

Содержание

1 Введение	2
2 Энтропия	4
2.1 Немного о степени неопределенности	4
2.2 Энтропия Шеннона и проверка ожидаемых свойств	6
3 Условная энтропия	11
3.1 Наводящие соображения	11
3.2 Определение условной энтропии	12
3.3 Пример на вычисление условной энтропии	17
3.4 Энтропия и прирост информации	19
3.5 Вычисление прироста информации на конкретном примере	21
4 Деревья принятия решений	23
4.1 Немного о самих деревьях	23
4.2 Алгоритм построения дерева и пример с кошками	25
4.3 Бинарное дерево решений	28
4.4 Типы признаков и их группировка	30
4.4.1 Алгоритм построения бинарного дерева решений и при-	
мер с кошками	32
4.4.2 Синтетический пример	34
5 Неопределенность Джини	37
5.1 Определение и свойства	37
5.2 Небольшое сравнение прироста Джини и энтропии	44
5.3 Прирост Джини на данных	45
6 Деревья принятия решений на реальном примере	46
7 Заключение	49

4 Деревья принятия решений

Итак, мы рассмотрели способ измерения прироста информации с использованием понятия энтропии. Теперь мы готовы перейти к построению дерева принятия решений, однако для начала неплохо бы определиться: а что это такое?

4.1 Немного о самих деревьях

Если вы знакомы с теорией графов, то определение дерева принятия решений может быть дано следующим образом.

Определение 4.1.1 *Дерево принятия решений – это дерево, то есть ациклический связный граф, имеющее следующие метки:*

- *узлы дерева, не являющиеся листьями – это атрибуты.*
- *в листьях дерева находятся отклики – результаты классификации.*
- *на ребрах находится правило – значение узла (атрибута), из которого исходит ребро.*

Даже если вы не знакомы с теорией графов, то введенное понятие прекрасно иллюстрируется на уже знакомом нам дереве принятия решений, цель которого – классифицировать предложения о работе на два типа: те, которые стоит принять, и те, которые стоит отклонить. Как видно из рисунка, из верхнего узла дерева, отвечающего за предиктор (или атрибут) «размер заработной платы», выходит три ребра. Если значение узла меньше, чем 50000, то мы сразу попадаем в лист: предложение предлагается отклонить. Если значение узла больше или равно, чем 100000, то мы снова попадаем в лист, но уже

с другим откликом: предложение предлагается принять. Если же значение узла находится в диапазоне от 50000 включительно до 100000 не включительно, то ребро нас ведет в следующий узел, который отвечает за время, требующееся на дорогу до работы. В зависимости от значения этого узла, возникают дополнительные ветвления, или ветки дерева, которые вы легко отследите самостоятельно. Обратите внимание, любой путь по построенному дереву заканчивается листом, и в каждый лист можно попасть из начальной (корневой) вершины дерева – это и есть требование связности в определении. Ацикличность же, или отсутствие циклов, обеспечивается тем, что как в каждый узел (кроме изначального), так и в каждый лист входит лишь одно ребро.

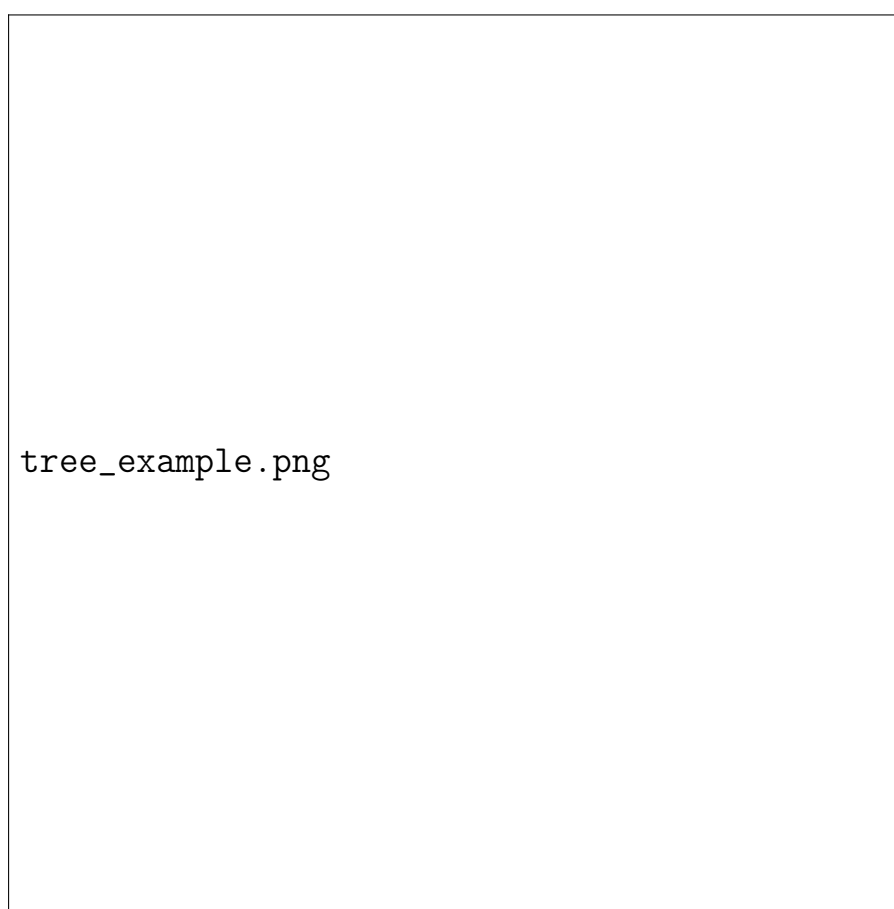


Рис. 2: Пример дерева решений.

Определение 4.1.2 *Глубиной конкретного листа (или узла) в дереве называется количество ребер, которые соединяют его с корневой вершиной. Глубиной дерева называют максимум из глубин его листов.*

В нашем примере всего 5 листов. Глубина каждого из них равна (слева направо): 1, 3, 3, 2 и 1, соответственно, а значит глубина дерева равна 3.

Разобравшись с тем, а что такое дерево принятия решений, а также как измерять информативность признаков, перейдем к рассмотрению конкретных алгоритмов построения деревьев принятия решений.

4.2 Алгоритм построения дерева и пример с кошками

Составим алгоритм построения дерева принятия решений на тренировочном наборе данных $x_1, x_2, \dots, x_n$, состоящем из n элементов с p предикторами $X_1, X_2, \dots, X_p$ каждый, и откликом Y .

1. Пусть на вход подается множество объектов X . Среди p предикторов выбрать тот, для которого прирост информации максимален. Итак, решается задача

$$\arg \max_{Q \in \{X_1, X_2, \dots, X_p\}} IG(Y|Q)$$

2. Пусть выбран предиктор X_i , принимающий на наборе данных X ровно t уникальных значений. Выполнить разделение набора данных X на подмножества $S_1, \dots, S_t$ по уникальным значениям предиктора X_i .
3. Для каждого множества S_i , $i \in \{1, 2, \dots, t\}$, если энтропия по отклику не равна нулю, повторить шаги 1 и 2.

Если использовать введенную выше терминологию деревьев принятия решений, то выбранный на первом шаге алгоритма предиктор X_i – это узел дерева, а разделение множества объектов на t подмножеств по уникальному значению предиктора – это установление t ребер из узла X_i . Равенство же нулю энтропии для некоторого множества S_i означает, что установленное ребро, в результате которого получилось множество S_i , указывает на лист, которому нужно присвоить значение откликов объектов из S_i (ведь раз энтропия ноль, то все объекты имеют одинаковый отклик).

Опробуем алгоритм на уже озвученном примере с кошками. Итак, таблица исходных данных в наших обозначениях такова:

№	Порода (X_1)	Шерсть (X_2)	Рост (X_3)	Привлекательность (Y)
x_1	Британец	Белый	Высокий	Нет
x_2	Британец	Серый	Высокий	Да
x_3	Британец	Белый	Низкий	Да
x_4	Мейн-кун	Белый	Высокий	Нет
x_5	Мейн-кун	Коричневый	Высокий	Да
x_6	Мейн-кун	Коричневый	Низкий	Нет
x_7	Рэгдолл	Серый	Высокий	Да
x_8	Мейн-кун	Серый	Низкий	Да

Иными словами, каждая строка таблицы – это тренировочный объект, обладающий тремя предикторами X_1, X_2, X_3 – «Порода», «Шерсть» и «Рост», соответственно, и откликом Y – «Привлекательность».

Первый шаг алгоритма нами уже был проделан ранее. Согласно разобранному примеру, самым информативным оказался критерий «Шерсть», или предиктор X_2 . Так как X_2 принимает три уникальных значения: «Белый», «Серый» и «Коричневый», то, согласно второму шагу алгоритма, весь набор тренировочных данных нужно разделить на 3 группы (по значению X_2). Пусть первая группа $S_{\text{Коричневый}}$ – группа кошек с коричневой шерстью, тогда в нее входят x_5, x_6 . Во вторую группу $S_{\text{Белый}}$, с белой шерстью, входят x_1, x_3, x_4 . В последнюю же группу $S_{\text{Серый}}$, с серой шерстью, входят все остальные, то есть x_2, x_7, x_8 . Давайте это визуализируем.

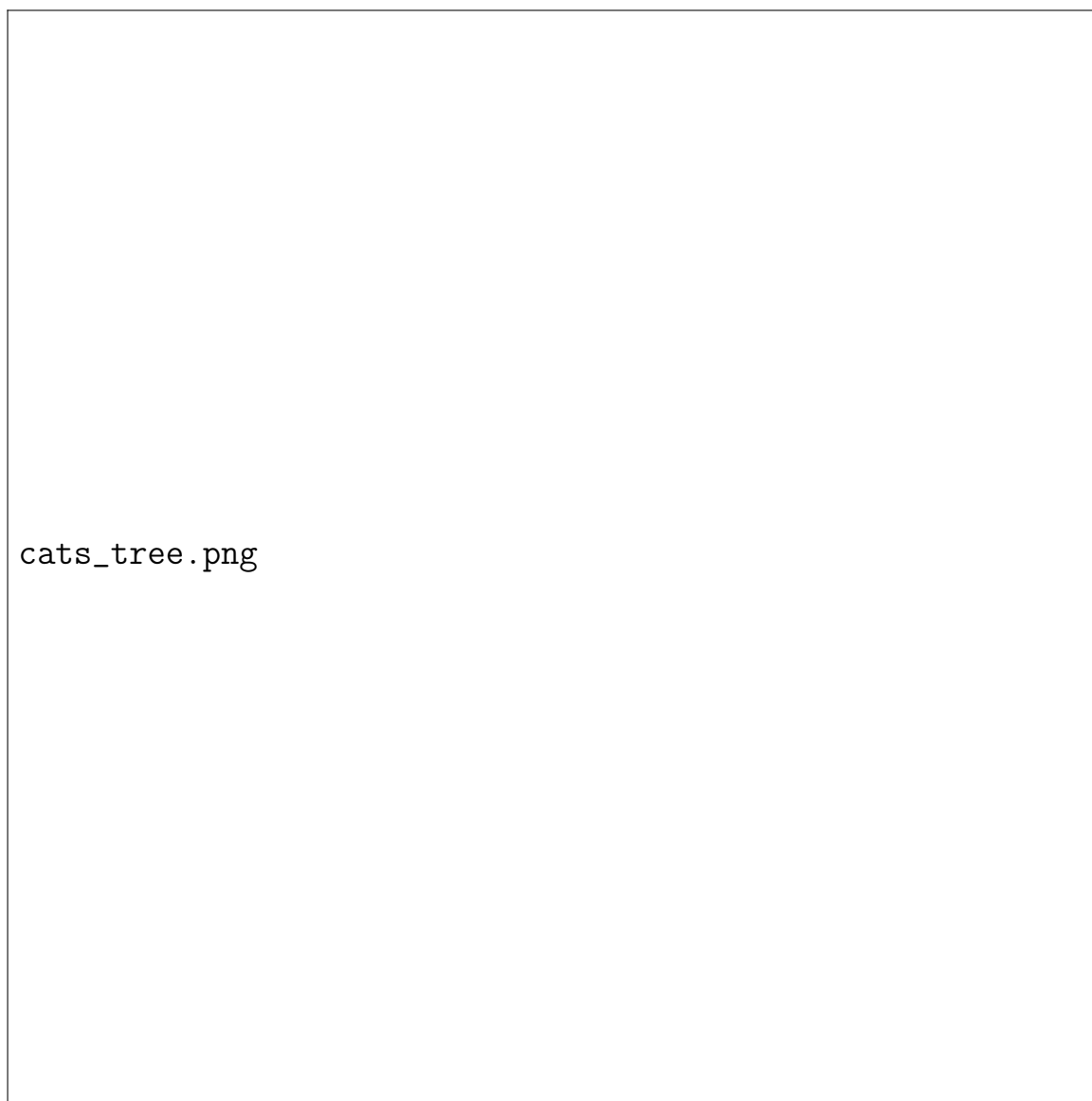


Рис. 3: Дерево принятия решений для задачи о кошках.

Посмотрите на рисунок [3](#) (он несколько отличается по визуализации

и информативности от ранее описанных деревьев). В верхнем белом блоке (корневом узле дерева) содержится информация об исходном наборе данных $X = \{x_1, x_2, \dots, x_n\}$. Во второй строке этого блока указана энтропия – это энтропия, вычисленная по отклику Y в исходном наборе данных. Она, как нами было показано, примерно равна 0.954. В третьей строке указано количество элементов исследуемого набора данных, а в четвертой – соотношение между привлекательными и не привлекательными кошками в исходном наборе данных (непривлекательных 3, привлекательных 5).

Так как «Шерсть» – самый информативный признак на первом этапе, принимающий 3 различных значения, то из корневого узла выходит три ребра, и появляется второй уровень дерева, состоящий из трех групп: $S_{\text{Коричневый}}$, $S_{\text{Белый}}$, $S_{\text{Серый}}$, отвечающих трем различным узлам. Так как среди кошек с коричневой шерстью 1 привлекательная и 1 нет, то энтропия по отклику «Привлекательность» в наборе данных $S_{\text{Коричневый}}$ равна 1. Аналогично, так как среди кошек с белой шерстью 2 непривлекательных и 1 привлекательная, то энтропия по отклику «Привлекательность» в наборе данных $S_{\text{Белый}}$ равна

$$-\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} \approx 0.918.$$

Среди кошек с серой шерстью все три оказались привлекательными, а потому энтропия по отклику «Привлекательность» в группе $S_{\text{Серый}}$ равна 0, и дальнейшее разбиение на подмножества группы $S_{\text{Серый}}$, в отличие от остальных двух групп, не требуется, а узел, соответствующий этому блоку, становится листом с откликом «Да».

В других же подмножествах остается неопределенность, а значит, в соответствии с алгоритмом, нужно определить новые критерии разбиения для каждого из них. Если внимательно посмотреть на подмножества $S_{\text{Коричневый}}$ и $S_{\text{Белый}}$ (ну или если просто посчитать), то становится понятно, что наиболее информативным критерием в каждой группе будет рост в холке. Отсюда и разделение на третьем уровне дерева. Легко видеть, что теперь каждое образовавшееся подмножество (их всего 4) имеет нулевую энтропию, значит алгоритм завершен, а все узлы последнего уровня становятся листьями с откликами (слева-направо) «Да», «Нет», «Нет» и «Да», соответственно.

Классификация новых объектов, имея построенное дерево, выполняется крайне просто. Новый объект сначала проходит классификацию по верхнему уровню (в нашем примере – по цвету шерсти). Если цвет шерсти серый, то дерево решений говорит, что кошка точно понравится судьям. Если же цвет коричневый или белый, то далее нужно смотреть на рост кошки (на следующий уровень). Согласно дереву, коричневая высокая или белая низкая кошка понравится судьям, а коричневая низкая или белая высокая – не понравится.

Замечание 4.2.1 Последний пункт описанного в начале алгоритма – это лишь одно из возможных условий останова процесса деления набора тренировочных данных на группы (и, как следствие, одно из возможных условий останова построения ДПР). Ситуация, когда энтропия каждой получившейся группы равна нулю означает, что в каждой группе все элементы имеют один и тот же отклик, а значит понятно и как эту группу «охарактеризовать» – по отклику. В то же время, особенно на больших объемах данных, разделение до нулевой энтропии – дело достаточно трудоемкое, зачастую приводящее к переобучению. Поэтому часто используют следующие критерии останова:

- ограничение глубины дерева. Узлы, имеющие максимальную установленную глубину, становятся листьями;
- ограничение минимального количества элементов в группе. Если при разделении рассматриваемого множества на подмножества по уникальным значениям предиктора, получающиеся группы тренировочных данных содержат меньше элементов, чем задано исследователем, то деление останавливается. В терминах ДПР это означает, что текущий узел становится листом;
- достижение в группе какого-то заданного критерия: например, значения неопределенности. В терминах ДПР это опять же означает, что текущий узел становится листом.

В каждом из описанных случаев, однако, остается открытым вопрос: какое же значение приписать листу? Обычно ему приписывают тот отклик, чьих представителей в соответствующей группе тренировочных данных большинство. Подробнее об этом мы поговорим чуть позже.

4.3 Бинарное дерево решений

Рассмотренный алгоритм построения дерева принятия решения удобен в том случае, когда мы работаем с предикторами, все возможные значения которых присутствуют в тренировочных данных. Если тестовый объект обладает значением предиктора, которого не было в тренировочных данных, то описанный алгоритм классификации, очевидно, не работает. Скажем, пусть нас интересует следующий вопрос: понравится ли судьям черная невысокая кошка? Взглянув на построенное ранее дерево (рисунок 3), мы понимаем, что уже на первом уровне наша классификация ломается: по какой ветке идти? Значение признака «Шерсть» не подходит ни под один из приведенных вариантов.

Озвученная проблема является, на самом деле, весьма серьезной. Дело в том, что нередко мы не знаем все возможные значения, которые может принимать атрибут рассматриваемых объектов, а часто бывает и так, что этих значений и вообще – бесконечное число (особенно, если значения атрибутов – числовые). Примерами числовых атрибутов, принимающих бесконечное число значений, могут служить: рост человека, текущее время, дата, масса товара на весах на рынке и многое-многое другое. Если же атрибут не числовой, он редко может принимать бесконечное число значений, но может принимать «очень много» значений. Примером могут служить названия городов по всему миру.

Кроме того, чем больше значений атрибутов мы имеем, тем дольше строится и само решающее дерево. К тому же, оно становится очень «ветвистым», что мешает как интерпретации модели в целом, так и способствует ее, модели, переобучению. Обозначенных причин, наверное, достаточно, чтобы понять, что требуется разработать какой-то новый подход к построению решающих деревьев, обладающий следующими преимуществами:

- возможность классификации объектов с ранее (то есть при обучении) не встречавшимися значениями предикторов;
- возможность объединять (группировать, укрупнять) множество значений признаков.

Выполнение последнего требования, как мы уже сказали, способствует уменьшению количества ветвлений и, как следствие, более простой интерпретируемости и более высокой скорости работы модели.

Все эти размышления приводят нас к так называемым бинарным деревьям принятия решений, или к бинарным решающим деревьям.

Определение 4.3.1 *Бинарное дерево принятия решений – это дерево принятия решений, из каждого узла которого выходит ровно два ребра.*

Итак, бинарные деревья принятия решений – это все те же деревья принятия решений, главная особенность которых состоит в том, что разделение осуществляется всегда на два подмножества, в каждом из которых заново рассчитывается энтропия и по проделанным расчетам определяется наиболее информативный критерий дальнейшего разбиения (если это разбиение требуется). Разделение же осуществляется по принципу: если значения атрибута удовлетворяют «установленному» условию, то объект относится к первому множеству, а если не удовлетворяют – ко второму. Таким образом, объекты, значения признаков которых ранее не встречались, все равно будут отнесены к одному из двух возможных подмножеств, а значит в итоге будут классифицированы. Но как проводить разделение? Что это за «установленные условия»?

4.4 Типы признаков и их группировка

Начнем с некоторых важных определений. Еще раз отметим, что мы работаем с тренировочным набором данных $x_1, x_2, \dots, x_n$ объема n , каждый объект которого обладает p атрибутами

$$x_i = (x_{i1}, x_{i2}, \dots, x_{ip}).$$

Определение 4.4.1 *Признак объекта называется категориальным, если он принимает конечное число значений.*

Категориальные признаки могут быть как числовыми, так и не числовыми. В рассмотренном примере с кошками все признаки – категориальные. Признак «Шерсть» принимает какое-то значение из конечного множества {Белый, Серый, Коричневый}, а «Рост» – из конечного множества {Высокий, Низкий}. Кроме категориальных, оказывается удобным выделять и некатегориальные признаки.

Определение 4.4.2 *Если признак объекта не является категориальным, то мы будем называть его некатегориальным.*

Примерами некатегориальных признаков, как мы уже упоминали ранее, являются: рост, текущее время, масса товара на весах в магазине и многие другие. Некатегориальные признаки чаще всего являются числовыми.

Теперь давайте решим, как можно объединять признаки в группы. Для некатегориального числового признака X_i часто поступают вот каким образом: множество его значений делят на два множества:

первое множество: $X_i \leq C$, второе множество: $X_i > C$,

причем число C (или так называемое пороговое значение) подбирается отдельно.

Замечание 4.4.1 *Давайте чуть подробнее поговорим про выбор числа C . Как этот выбор осуществляется? Тут возможно много вариантов, мы же приведем лишь несколько.*

1. *Первый вариант такой. Для начала определяют множество значений (обычно отрезок) $[a, b]$ некатегориального числового признака X_i либо исходя из каких-то теоретических соображений, либо исходя из значений этого признака у тренировочных данных. В последнем случае в качестве a часто берут наименьшее значение рассматриваемого признака среди тренировочных данных, а в качестве b – наибольшее. Следующий этап – это задание шага $h > 0$ изменения значения C .*

При выбранном шаге h , в качестве значений C разумно рассмотреть следующие:

$$\{a, a + h, \dots, a + (k - 1)h, a + kh\}, \quad k \in \mathbb{Z},$$

где $a + (k - 1)h < b$, но $a + kh \geq b$. Для каждого из выбранных значений вычисляют прирост информации, отвечающий полученному разделению на два класса: $X_i \leq C$ и $X_i > C$.

2. Вторым вариантом похож на первый. Значения C выбираются таким образом, чтобы при каждом следующем значении C к множеству $X_i \leq C$ добавлялось ровно одно новое значение предиктора X_i . Формально это можно описать следующим образом. Пусть $\{x_{1i}, x_{2i}, \dots, x_{ti}\}$, $1 \leq t \leq n$ – множество всех возможных различных значений предиктора X_i на тренировочных данных объема n . Предположим также, что они упорядочены по возрастанию, то есть

$$x_{1i} < x_{2i} < \dots < x_{ti}.$$

Тогда в качестве значений C разумно взять следующие:

$$\{C_1, C_2, \dots, C_t\},$$

где $C_t \geq x_{ti}$, $x_{ji} \leq C_j < x_{(j+1)i}$, $j \in \{1, 2, \dots, (t - 1)\}$.

В итоге выбирают то разделение, при котором прирост информации наибольший.

Если же признак X_i является категориальным и принимает значения из множества $M = \{x_{1i}, x_{2i}, \dots, x_{ti}\}$, $t \in \{1, 2, \dots, n\}$, то его множество значений можно разделить на две части, например, так:

первое множество: $X_i \in M_1$, второе множество: $X_i \in M_2$,

где $M_1 \cap M_2 = \emptyset$, $M_1 \cup M_2 = M$, $M_j \neq \emptyset$, $j \in \{1, 2\}$. Иными словами, все множество M уникальных значений признака X_i разбивается на 2 непустые непересекающиеся части.

Замечание 4.4.2 В библиотеках, реализованных в различных инструментах, чаще всего множество M_1 составляется ровно из одного значения признака X_i , тогда как M_2 составляется из всех остальных значений. Тем самым, если $M = \{x_{1i}, x_{2i}, \dots, x_{ti}\}$, $t \in \{1, 2, \dots, n\}$, то

$$M_1 = \{C\}, \quad M_2 = M \setminus M_1,$$

где на каждой итерации C принимает по очереди значения каждого элемента множества M . В итоге выбирают то разделение, при котором прирост информации наибольший.

4.4.1 Алгоритм построения бинарного дерева решений и пример с кошками

Составим алгоритм построения бинарного дерева для тренировочного набора данных $X = \{x_1, x_2, \dots, x_n\}$ объема n с p предикторами $X_1, X_2, \dots, X_p$ и откликом Y .

1. Пусть на вход подается множество объектов X . Среди p предикторов выбрать тот, для фиксированного разбиения значений которого на подмножества достигается наибольший прирост информации. Итак, решается задача

$$\arg \max_{Q \in \{X_1, X_2, \dots, X_p\}} \text{IG}(Y|Q(C)),$$

где C определяется в зависимости от типа признака, согласно описанным выше правилам.

2. Пусть выбран предиктор X_i и найдено C . Выполнить разделение по выбранному признаку на два подмножества S_C и $\bar{S}_C$. В первое подмножество входят объекты, значения i -ого атрибута которых удовлетворяют критерию разделения, во второе – все остальные.
3. Повторить шаги 1 и 2 для множеств S_C и $\bar{S}_C$ либо согласно с ограничением числа уровней (согласно замечанию 4.2.1), либо до нулевой энтропии в каждой из групп, если ограничение не задано.

Опробуем алгоритм все на тех же кошках. Так, согласно выполненным ранее вычислениям, предиктор «Шерсть» со значением «Серый» обеспечивал нулевую энтропию (то есть максимальный прирост информации). Этот критерий и будет первым при формировании двух подмножеств $S_{\text{Серый}}$ и $\bar{S}_{\text{Серый}}$. В первое множество войдут все кошки с серой шерстью, во второе – все остальные. Энтропия во втором множестве составит

$$-\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} \approx 0.971.$$

В первом подмножестве неопределенности нет, а вот во втором неопределенность сохраняется. Согласно третьему шагу алгоритма это означает, для этого подмножества необходимо повторить первые два шага алгоритма. Дальнейшее разбиение подмножества и критерии можно увидеть на рисунке 4. Оказывается, на втором уровне самым информативным снова является разделение кошек по цвету, но теперь уже на «Белых» и «Не белых». После этого, на последнем уровне, кошки делятся в зависимости от размера. Легко видеть, что на последнем уровне дерева все подмножества имеют нулевую энтропию, что означает, что построение завершено.

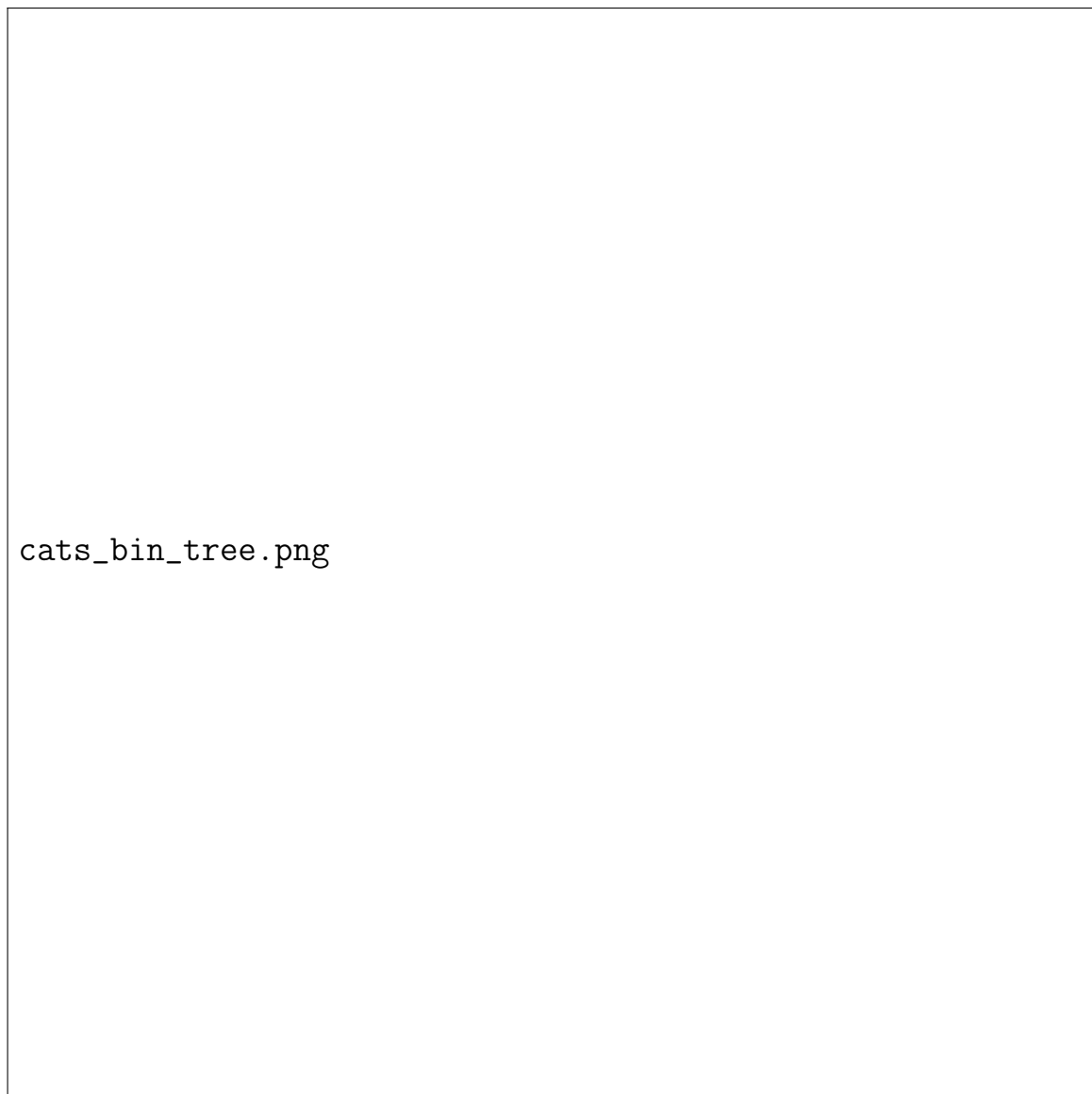


Рис. 4: Пример бинарного дерева решений.

Бинарное дерево, как мы уже отмечали, позволяет проводить классификацию новых объектов даже в случае, когда значение какого-то предиктора оказалось уникальным (или новым). Так, классификация невысокой черной кошки в случае дерева принятия решений, построенного ранее, была невозможна. В то же время, если рассмотреть только что построенное бинарное дерево, классификация возможна. Так как черная кошка – не серая, то идем по левой ветке. Опять, так как черная кошка – не белая, снова идем по левой ветке. Ну и, наконец, так как рассматриваемая кошка – не высокая, то идем по правой ветке и делаем вывод, что кошка не понравится судьям.

4.4.2 Синтетический пример

Опробуем алгоритм на синтетических данных с двумя предикторами X_1 и X_2 , причем на таких, которые интуитивно разбиваются прямой на два подмножества. Предикторы, отвечающие рассматриваемым данным, принимают лишь числовые значения.

Задача все та же – провести двухклассовую классификацию. У данных с откликом 0 (зеленые) предикторы независимы и имеют распределение $N_{0,1}$. Таких точек сгенерировано 100 штук. У данных с откликом 1 (желтые) предикторы тоже независимы, но имеют распределение $N_{2,1}$. Всего сгенерировано 200 значений, они изображены на рисунке [5](#).

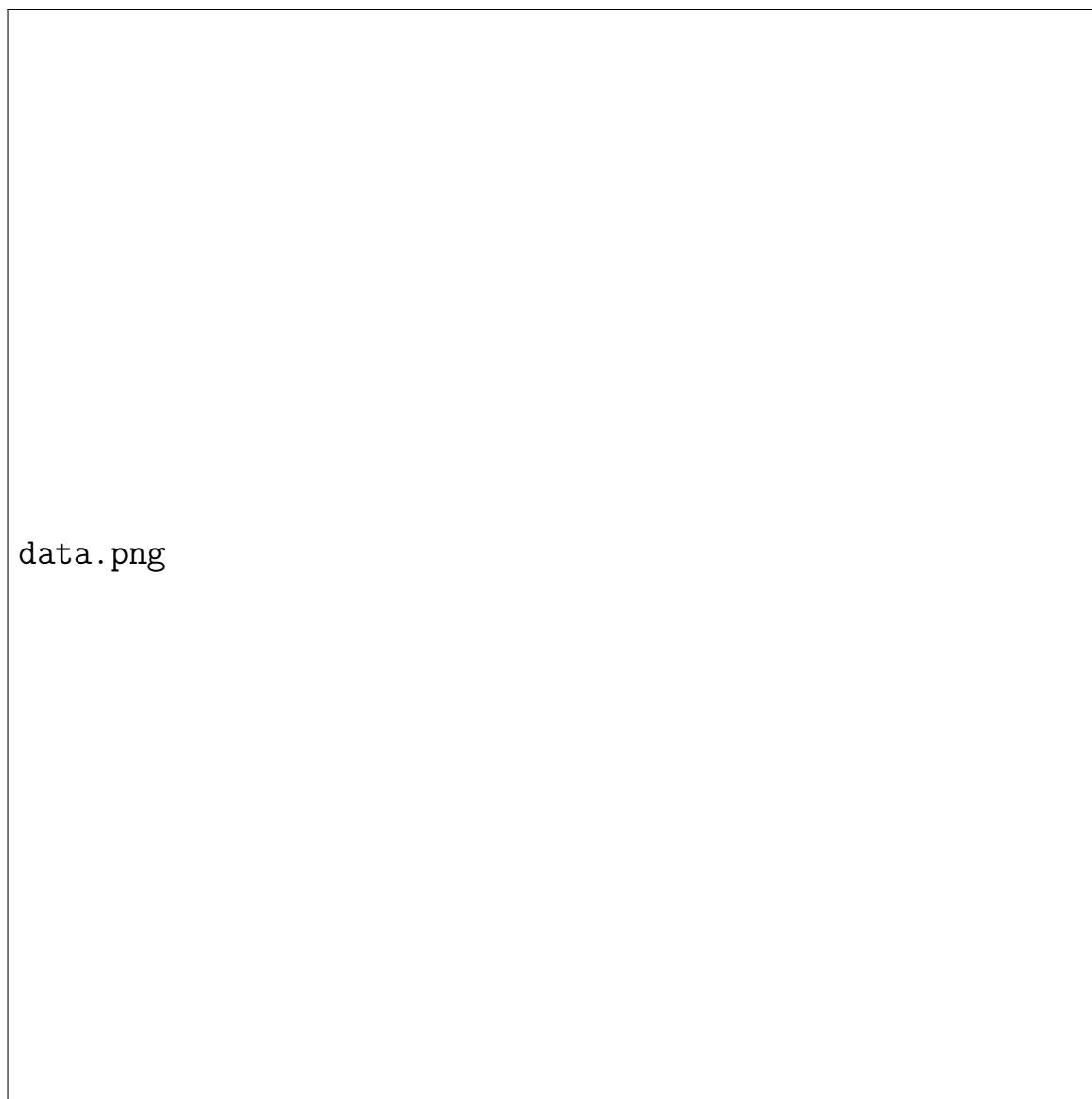


Рис. 5: Объекты интуитивно делимы.

Построим бинарное решающее дерево, ограничившись лишь двумя уровнями (деревом глубины 2). Начальная энтропия рассматриваемого набора

данных, конечно, максимальна и равна единице, так как у нас поровну единиц и нулей в отклике. Расчет в системе моделирования привел к тому, что для начального набора разделение будет оптимальным по признаку $X_2 \leq 0.77$. Таким образом, выполняются действия, аналогичные рассмотренному примеру с породами кошек, однако заметно возрастает количество вычислений.

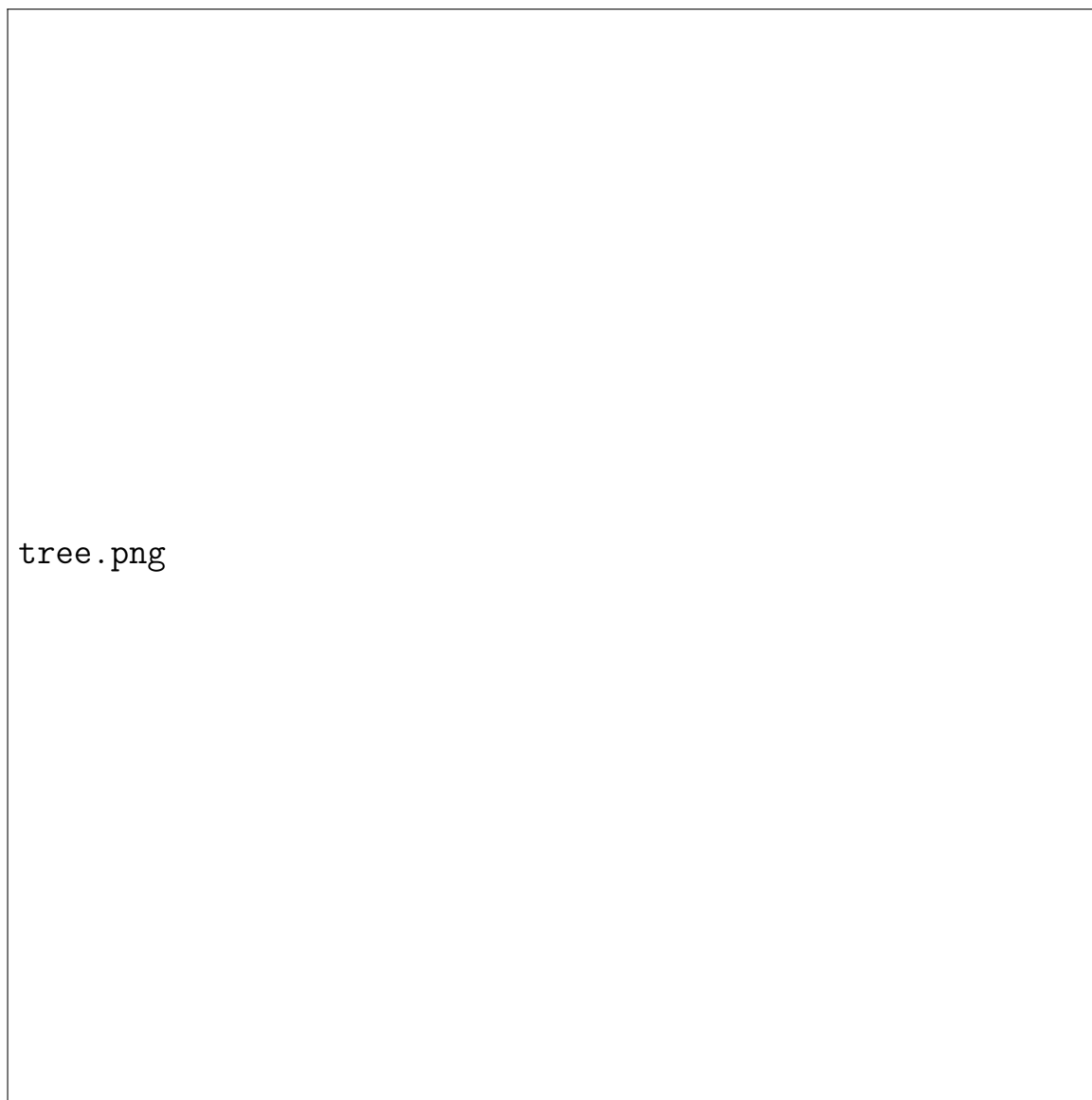


Рис. 6: Дерево решений.

На рисунке [6](#) отражено сказанное нами ранее: исходный набор из 200 элементов делится на два по признаку $X_2 \leq 0.77$. При этом к зеленому блоку относятся те данные, которые удовлетворяют условию разделения, то есть данные, второй признак которых не превосходит 0.77, а к красному – все остальные. Так как энтропия в каждой группе отлична от нуля, то алгоритм продолжает свою работу. Видно, что в обеих группах следующее разделение идет по признаку X_1 . Левая группа делится по условию $X_1 \leq 1.59$, а правая – по условию $X_1 \leq 0.04$.

В данном примере глубина ограничена 2 уровнями, но, как мы видим, даже у такого «ограниченного» дерева есть группа с нулевой энтропией (самая левая), что, конечно, хорошо. Согласно сформулированному алгоритму, все узлы на втором уровне дерева становятся листьями с откликами «Зеленый», «Желтый», «Зеленый», «Желтый», соответственно. Например, почему второй лист отвечает отклику «Желтый»? Как видно, из всех тренировочных данных в этот блок попадает 3 зеленых и 7 желтых объектов. Отклик же устанавливается по отклику наибольшего числа входящих объектов. Так как объектов с откликом «Желтый» большинство, то таков отклик и у листа.

Классификацию (а также сформулированные условия разделения) при помощи ДПР в нашем примере можно визуализировать непосредственно на плоскости. Для этого на первом этапе всю плоскость достаточно разделить прямой $X_2 = 0.77$. Мы видим на рисунке 7, что плоскость разделилась на две части: нижняя, отвечающая левому блоку в дереве ($X_2 \leq 0.77$) и верхняя, отвечающая правому блоку ($X_2 > 0.77$).

Далее, условие следующего разделения для нижней части плоскости (левого блока в дереве) – это условие $X_1 \leq 1.59$, а для верхней (или правого блока) – условие $X_1 \leq 0.04$. Визуализацию разделения вы видите на рисунке 8.

На этом этапе дерево считается обученным. Мы видим, что оно допускает ошибки и на тренировочных данных – это ситуации, когда желтые точки оказываются в зеленой области, или, наоборот, когда зеленые точки попадают в желтую область; всего таких ошибок – 17. Посмотрим, как наше дерево будет работать на новой выборке из нормального распределения с такими же параметрами. Как видно, в целом, число ошибок сохранилось: на этапе обучения их было 17, а на этапе тестирования – 21. На интуитивном уровне, дерево ошибается примерно в 11% случаев, что не так уж и плохо, а значит дерева с двумя уровнями вполне достаточно. Но что произойдет, если увеличить количество уровней, например, до шести? Конечно, дерево станет заметно больше, но и число групп с нулевой энтропией возрастет. На рисунке изображены области классификации, отвечающие условиям деления на группы при построении дерева на все тех же исходных данных, как и при построении дерева с двумя уровнями.

Похоже, исходя из наглядного представления дерева, что мы получили переобучение: и число областей, и их причудливая форма кажутся совершенно неестественными. И это действительно так. Протестировав наше дерево на тестовом наборе, мы видим, что имеются как пустые области, которые не используются вовсе (в них не попадает ни одно из тестовых данных), так и области, в которые теперь попадает много точек из другого класса – ошибки классификации.



Рис. 7: Классификация на основе дерева решений (первое разделение).